

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий інститут прикладного системного аналізу

ЗВІТ

про виконання лабораторної роботи №2
з дисципліни «Обробка надвеликих масивів даних»

на тему: **«Робота з графовими структурами»**

Варіант 3: Німеччина – Велика Британія

Виконав:
студент групи КІ-61мп
Гавлицький Іван

Перевірив:
Пишнограєв Іван Олександрович

ЗМІСТ

МЕТА РОБОТИ	3
1 ОПИС ЗАДАЧІ ТА ВХІДНИХ ДАНИХ	3
1.1 Постановка задачі	3
1.2 Вхідні дані	4
2 ОПИС РЕАЛІЗАЦІЇ	5
2.1 Програмне середовище	5
2.2 Завантаження та очищення даних	7
2.3 Обґрунтування вибору алгоритмів	13
2.4 Завдання 1. Сумарна довжина рейсів авіакомпаній	14
2.5 Завдання 2. Пошук маршрутів Німеччина – Велика Британія	15
2.6 Завдання 3. Найкоротший та найдовший маршрути TXL – EDI	21
2.7 Завдання 4. PageRank	23
2.8 Завдання 5. Кластеризація та аналіз зв'язності	26
3 АНАЛІЗ РЕЗУЛЬТАТІВ	28
3.1 Авіакомпанія з найбільшим кілометражем	28
3.2 Порівняння пошуку мотивів і BFS	30
3.3 Найвпливовіші аеропорти за PageRank	32
3.4 Кластери та ізольовані підграфи	34
ВИСНОВКИ	38
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	39
ДОДАТОК А. ПРОГРАМНИЙ КОД	41
ДОДАТОК Б. ДОПОМІЖНІ МОДУЛІ	41

МЕТА РОБОТИ

Ознайомитися з бібліотеками для обробки графів Spark GraphX та GraphFrames, вивчити основні алгоритми центральності та пошуку шляхів. Набути практичних навичок аналізу зв'язків у великих масивах даних, використовуючи набори даних авіап перевезень OpenFlights. Навчитися виконувати з'єднання даних (Join) для підготовки атрибутів вершин та ребер графа.

1 ОПИС ЗАДАЧІ ТА ВХІДНИХ ДАНИХ

1.1 Постановка задачі

Відповідно до завдання необхідно:

1. завантажити дані OpenFlights у DataFrame, з'єднанням (Join) додати до маршрутів координати аеропортів, обчислити відстань кожного рейсу за формулою гаверсинуса та знайти авіакомпанії з найбільшою і найменшою сумарною довжиною рейсів;
2. побудувати в Spark GraphFrames граф (вершини – аеропорти, ребра – рейси) і знайти всі маршрути між країнами варіанта з не більше ніж двома пересадками (до трьох сегментів), використавши пошук мотивів і BFS, та порівняти їх за результатами і складністю реалізації;
3. знайти найкоротший і найдовший за фізичною відстанню (км) маршрут між двома аеропортами країн варіанта з не більше ніж трьома пересадками;
4. знайти алгоритмом PageRank 5 найвпливовіших аеропортів, порівняти їх з найбільшими хабами світу та проаналізувати вплив параметра `resetProbability`;

5. виділити кластери аеропортів (Connected Components або Label Propagation), проаналізувати кластери з не менше ніж 5 аеропортів і знайти ізольовані підграфи з не менше ніж 2 вузлів.

Варіант 3 сформульовано як «Німеччина – Велика Британія». Маршрути в OpenFlights напрямлені, тому в завданні 2 його інтерпретовано як пошук маршрутів в одному напрямку: з аеропортів Німеччини до аеропортів Великої Британії. Зворотний напрямок не обчислювався. Для завдання 3 обрано пару аеропортів цих країн: Берлін-Тегель (TXL) – Единбург (EDI).

1.2 Вхідні дані

Використано відкриті дані проекту OpenFlights [1]. Файли завантажуються з репозиторію проекту на GitHub, зафіксованого на одному коміті [2], тому кожен запуск отримує ті самі дані. Структуру файлів наведено в табл. 1.1. Файли не мають рядка заголовка, відсутні значення позначено як \N.

Таблиця 1.1 – Структура вхідних даних

Файл	Записів	Основні поля
airports-extended.dat	12 668	id, name, city, country, iata, icao, latitude, longitude, type
airlines.dat	6 162	id, name, iata, icao, country, active
routes.dat	67 663	airline, airline_id, src, src_id, dst, dst_id, codeshare, stops, equipment

За описом джерела [1], база маршрутів відображає стан на червень 2014 року: 67 663 маршрути між 3 321 аеропортом. Маршрути напрямлені: якщо авіакомпанія літає з А до В і назад, це два записи. Поле codeshare = 'Y' позначає рейс, який виконує інший перевізник, а продає ця авіакомпанія під своїм кодом. Частоти рейсів, кількість місць і пасажиропотік у даних від-

сутні: маршрут лише вказує, що авіакомпанія сполучає два аеропорти. Файл `airports-extended.dat`, на відміну від `airports.dat`, містить також залізничні станції, поромні термінали та записи, додані користувачами.

2 ОПИС РЕАЛІЗАЦІЇ

2.1 Програмне середовище

Роботу виконано в ноутбучі Jupyter (див. додаток А) на кластері Apache Spark 4.0.4 у режимі Standalone [3], розгорнутому локально в Docker Compose. Склад середовища наведено в табл. 2.1, схему взаємодії компонентів – на рис. 2.1.

Таблиця 2.1 – Програмне середовище

Компонент	Призначення та параметри
spark-master	менеджер ресурсів кластера, <code>spark://127.0.0.1:7077</code>
spark-worker-1, -2	виконавці (executors): по 4 ядра та 6 ГБ пам'яті, Python 3.13 для коду на Python
Драйвер	процес ядра Jupyter: PySpark 4.0.4, GraphFrames 0.12.3, pandas
Дані	директорія <code>data/</code> , змонтована за тим самим шляхом у кожен контейнер

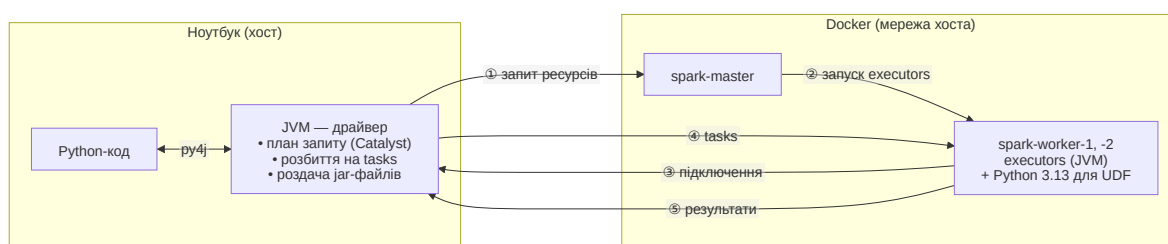


Рисунок 2.1 – Архітектура кластера

Ноутбук не є клієнтом кластера, а виконує роль драйвера: PySpark запускає локальну JVM, яка будує план кожного запиту оптимізатором Catalyst і ділить його на задачі. Master лише виділяє ресурси (кроки 1–2 на рис. 2.1); executors у контейнерах підключаються до драйвера, самі читають дані з data/ і обмінюються проміжними результатами (shuffle) (кроки 3–5). Контейнери використовують мережу хоста, тому executors звертаються до драйвера за адресою 127.0.0.1. Стан контейнерів після запуску показано на рис. 2.2, вебінтерфейс master – на рис. 2.3: два робочі вузли по 4 ядра, і всі 8 ядер зайняті застосунком lab002 (сесією ноутбука).

```
$ docker compose --progress quiet -f infra/docker-compose.yml up -d --build --wait --wait-timeout 300
$ docker compose --progress quiet -f infra/docker-compose.yml ps --format 'table {{.Name}}      {{.Status}}'
NAME                STATUS
spark-master        Up 13 seconds (healthy)
spark-worker-1       Up 6 seconds (healthy)
spark-worker-2       Up 6 seconds (healthy)
```

Рисунок 2.2 – Контейнери кластера після запуску



Spark Master at spark://127.0.0.1:7077

URL: spark://127.0.0.1:7077

Workers: 2 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 8 Total, 8 Used

Memory in use: 12.0 GiB Total, 2.0 GiB Used

Resources in use:

Applications: 1 [Running](#), 0 [Completed](#)

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE ([Environment](#), [Log](#))

Workers (2)

Worker Id	Address	State	Cores	Memory
worker-20261008160810-127.0.0.1-41013	127.0.0.1:41013	ALIVE	4 (4 Used)	6.0 GiB (1024.0 MiB Used)
worker-20261008160810-127.0.0.1-41575	127.0.0.1:41575	ALIVE	4 (4 Used)	6.0 GiB (1024.0 MiB Used)

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261008160831-0000 (kill)	lab002	8	1024.0 MiB		2026/10/08 16:08:31	nuinashco	RUNNING	56 min

Рисунок 2.3 – Вебінтерфейс Spark Master: робочі вузли та запущений застосунок

2.2 Завантаження та очищення даних

Файли завантажує скрипт `download_data.py` (додаток Б, рис. 2.4). Оскільки файли не мають заголовків, схеми задано явно, а значення `\N` читаються як `NULL` (лістинг 2.1). Шлях до файлів абсолютний: файл читають `executors`, а директорія `data/` змонтована за цим шляхом на кожному вузлі. Кількість записів показано на рис. 2.5, приклад маршрутів – на рис. 2.6.

```
$ /home/nuinashco/Projects/iasa-bigdata/.venv/bin/python scripts/download_data.py
[download] airports-extended.dat: 12,668 lines, 1.7 MB
[download] airlines.dat: 6,162 lines, 0.4 MB
[download] routes.dat: 67,663 lines, 2.4 MB
```

Рисунок 2.4 – Завантаження даних OpenFlights

Лістинг 2.1 – Читання файлів OpenFlights зі схемами

```

S, I, D = StringType(), IntegerType(), DoubleType()

def schema(*fields):
    return StructType([StructField(name, dtype) for name, dtype in fields])

AIRPORTS = schema(
    ("id", I), ("name", S), ("city", S), ("country", S), ("iata", S), ("icao",
    S), ("latitude", D),
    ("longitude", D), ("altitude", I), ("timezone", D), ("dst", S), ("tz", S),
    ("type", S), ("source", S),
)

AIRLINES = schema(
    ("id", I), ("name", S), ("alias", S), ("iata", S), ("icao", S), ("callsign",
    S), ("country", S), ("active", S),
)

ROUTES = schema(
    ("airline", S), ("airline_id", I), ("src", S), ("src_id", I), ("dst", S),
    ("dst_id", I), ("codeshare", S),
    ("stops", I), ("equipment", S),
)

def read(name, file_schema):
    # Absolute path: executors read the file themselves, and data/ is mounted
at this path on every worker.
    return spark.read.csv(str(OPENFLIGHTS / name), schema=file_schema,
        nullValue="\\N")

airports = read("airports-extended.dat", AIRPORTS)
airlines = read("airlines.dat", AIRLINES)
routes = read("routes.dat", ROUTES)
for name, df in [("airports", airports), ("airlines", airlines), ("routes",
    routes)]:
    df.createOrReplaceTempView(name)

```

rows	
airports	12668
airlines	6162
routes	67663

Рисунок 2.5 – Кількість записів у таблицях

	airline	airline_id	src	src_id	dst	dst_id	codeshare	stops	equipment
0	2B	410	AER	2965	KZN	2990	None	0	CR2
1	2B	410	ASF	2966	KZN	2990	None	0	CR2
2	2B	410	ASF	2966	MRV	2962	None	0	CR2
3	2B	410	CEK	2968	KZN	2990	None	0	CR2
4	2B	410	CEK	2968	OVV	4078	None	0	CR2

Рисунок 2.6 – Перші маршрути таблиці routes

Перевірку якості маршрутів наведено в лістингу 2.2 і на рис. 2.7: 423 маршрути не мають ідентифікатора аеропорту (з'єднання з координатами для них неможливе), 479 – ідентифікатора авіакомпанії, 14 597 є codeshare-маршрутами, а один маршрут є петлею (аеропорт вильоту збігається з аеропортом прильоту).

Лістинг 2.2 – Перевірка якості маршрутів

```

SELECT
    COUNT(*) AS routes,
    COUNT_IF(src_id IS NULL OR dst_id IS NULL) AS without_airport_id,
    COUNT_IF(airline_id IS NULL) AS without_airline_id,
    COUNT_IF(codeshare = 'Y') AS codeshare,
    COUNT_IF(src_id = dst_id) AS same_airport

FROM
    routes

```

	routes	without_airport_id	without_airline_id	codeshare	same_airport
0	67663	423	479	14597	1

Рисунок 2.7 – Проблеми якості в таблиці маршрутів

Відстань по великому колу між аеропортами вильоту (φ_1, λ_1) і прильоту (φ_2, λ_2) обчислюється за формулою гаверсинуса:

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\varphi_2 - \varphi_1}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\lambda_2 - \lambda_1}{2}}, \quad R = 6371 \text{ км.} \quad (2.1)$$

Представлення `route_distances_all` (лістинг 2.3) з'єднує маршрути з таблицею аеропортів двічі – для аеропорту вильоту та прильоту – і обчислює відстань (2.1). `INNER JOIN` відкидає маршрути без ідентифікатора аеропорту, умова `WHERE` – петлю.

Лістинг 2.3 – Координати маршрутів і відстань за формулою гаверсинуса

```

CREATE OR REPLACE TEMP VIEW route_distances_all AS
SELECT
    r.airline,
    r.airline_id,
    r.codeshare,
    r.src_id,
    r.dst_id,
    s.iata AS src_iata,
    d.iata AS dst_iata,
    s.country AS src_country,
    d.country AS dst_country,
    2 * 6371 * ASIN(SQRT(
        POWER(SIN(RADIANS(d.latitude) - s.latitude) / 2), 2)
        + COS(RADIANS(s.latitude)) * COS(RADIANS(d.latitude))
        * POWER(SIN(RADIANS(d.longitude) - s.longitude) / 2), 2)
    )) AS distance_km

FROM
    routes AS r

INNER JOIN
    airports AS s
    ON r.src_id = s.id

INNER JOIN
    airports AS d
    ON r.dst_id = d.id

WHERE 1=1
    AND r.src_id != r.dst_id

```

Сортування за відстанню виявило пошкоджений запис (рис. 2.8, 2.9): найдовші «рейси» (понад 15 900 км, довше за будь-який реальний безпосадковий рейс) ведуть до аеропорту 5613, у якого місто й країна – Солвезі, Замбія, а назва, код ICAO та координати належать аеродрому Los Alamitos

у Каліфорнії. Маршрути через цей аеропорт виключено (лістинг 2.4); наступний за довжиною маршрут SYD → DFW (13 808 км) реальний.

	airline	src_id	src_iata	dst_id	dst_iata	distance_km
0	P0	907	LUN	5613	NaN	16082.0
1	P0	5613	NaN	907	LUN	16082.0
2	P0	910	NLA	5613	NaN	15937.0
3	P0	5613	NaN	910	NLA	15937.0
4	AA	3361	SYD	3670	DFW	13808.0
5	QF	3361	SYD	3670	DFW	13808.0

Рисунок 2.8 – Найдовші маршрути до очищення

	id	name	city	country	icao	latitude	longitude	tz
0	5613	Los Alamitos Army Air Field	Solwesi	Zambia	KSLI	33.790001	-118.052002	America/Los_Angeles

Рисунок 2.9 – Пошкоджений запис аеропорту 5613

Лістинг 2.4 – Виключення маршрутів через пошкоджений аеропорт

```
CREATE OR REPLACE TEMP VIEW route_distances AS
SELECT
    *
FROM
    route_distances_all
WHERE 1=1
    AND src_id NOT IN (5613)
    AND dst_id NOT IN (5613)
```

Після очищення залишилося 67 235 маршрутів з координатами (рис. 2.10) довжиною від 1 до 13 808 км, у середньому 1 845 км.

	routes	min_km	avg_km	max_km
0	67235	1.0	1845.0	13808.0

Рисунок 2.10 – Статистика відстаней після очищення

2.3 Обґрунтування вибору алгоритмів

Модель графа. Вершини – аеропорти, що мають хоча б один маршрут, ребра – напрямки «аеропорт → аеропорт» з атрибутами `distance_km` і кількістю авіакомпаній. Маршрути різних авіакомпаній між тією ж парою аеропортів об'єднано в одне ребро: інакше кожен шлях повторювався б для кожної комбінації перевізників, а пошук маршрутів вимірює зв'язність мережі, а не кількість перевізників. Граф напрямлений, як і маршрути OpenFlights.

Пошук мотивів (завдання 2, 3). Завдання вимагають усіх маршрутів до заданої кількості пересадок і їх фізичної довжини. Мотив $(a) - [e1] - > (b)$; $(b) - [e2] - > (c)$ [4] перетворюється на з'єднання таблиць ребер і вершин, тому дає кожен шлях заданої довжини з атрибутами всіх ребер, з яких сумується відстань. Для мотивів написано допоміжну функцію `simple_paths` (додаток Б): вона будує ланцюжок з k ребер, додає умови на першу й останню вершину та забороняє повторне відвідування аеропорту (простий шлях).

BFS (завдання 2). Пошук у ширину – рекомендований завданням альтернативний метод; його виконано для порівняння з мотивами. Він знаходить шляхи з мінімальною кількістю сегментів, але не всі шляхи до заданої довжини і без урахування кілометрів, тому для завдання 3 не підходить.

PageRank (завдання 4) [5] оцінює транзитивний вплив: ранг аеропорту залежить не лише від кількості напрямків, а й від рангу аеропортів, з яких до нього летять. Для порівняння обчислено ступеневу центральність (кількість напрямків).

Connected Components і Label Propagation (завдання 5). Слабко зв'язні компоненти (напрямок ребер ігнорується) прямо відповідають на питання про ізольовані підграфи: група аеропортів, не досяжна з основної мережі, – окрема компонента. Проте майже всі аеропорти утворюють одну

компоненту, тому для виділення кластерів усередині мережі додатково застосовано Label Propagation: кожна вершина ітеративно переймає найчастішу мітку сусідів, і тісніше пов'язані між собою аеропорти отримують спільну мітку.

2.4 Завдання 1. Сумарна довжина рейсів авіакомпаній

Сумарну довжину рейсів обчислює запит з лістингу 2.5. Враховано лише маршрути, які авіакомпанія виконує сама: codeshare-маршрут – це рейс іншого перевізника. Кожен маршрут рахується один раз, незалежно від частоти рейсів. Результати для порядку DESC і ASC наведено на рис. 2.11, 2.12.

Лістинг 2.5 – Сумарна довжина маршрутів авіакомпаній

```
SELECT
    a.name AS airline,
    a.country,
    COUNT(*) AS routes,
    ROUND(SUM(f.distance_km)) AS total_km

FROM
    route_distances AS f

INNER JOIN
    airlines AS a
    ON f.airline_id = a.id

WHERE 1=1
    AND f.codeshare IS NULL

GROUP BY a.id, a.name, a.country

ORDER BY total_km DESC

LIMIT 10
```

	airline	country	routes	total_km
0	Ryanair	Ireland	2484	3700425.0
1	US Airways	United States	1446	3477460.0
2	American Airlines	United States	1265	3342406.0
3	United Airlines	United States	905	3046248.0
4	Delta Air Lines	United States	1146	3013769.0
5	China Southern Airlines	China	1238	1733470.0
6	easyJet	United Kingdom	1130	1582234.0
7	Southwest Airlines	United States	1098	1542333.0
8	Air China	China	786	1521810.0
9	Lufthansa	Germany	507	1513728.0

Рисунок 2.11 – Авіакомпанії з найбільшою сумарною довжиною маршрутів

	airline	country	routes	total_km
0	TransHolding System	Brazil	2	36.0
1	SOCHI AIR CHATER	Russia	2	38.0
2	Air Seychelles	Seychelles	2	87.0
3	LACSA	Costa Rica	2	111.0
4	Reem Air	Kyrgyzstan	2	142.0
5	Sibaviatrans	Russia	2	170.0
6	International AirLink	Jamaica	2	179.0
7	Papillon Grand Canyon Helicopters	United States	2	186.0
8	Helijet	Canada	2	198.0
9	40-Mile Air	United States	2	237.0

Рисунок 2.12 – Авіакомпанії з найменшою сумарною довжиною маршрутів

2.5 Завдання 2. Пошук маршрутів Німеччина – Велика Британія

Граф будується з представлення `route_distances` (лістинг 2.6) і містить 3 329 вершин і 37 269 ребер (рис. 2.13). Обидва `DataFrame` кешуються, бо кожен наступний алгоритм читає їх багато разів.

Лістинг 2.6 – Побудова графа авіасполучень

```
edges = spark.sql(r"""
SELECT
    src_id AS src,
    dst_id AS dst,
```

```

    FIRST(distance_km) AS distance_km,
    COUNT(DISTINCT airline_id) AS airlines

FROM
    route_distances

GROUP BY src_id, dst_id
""").cache()
edges.createOrReplaceTempView("edges")

vertices = spark.sql(r"""
SELECT
    a.id,
    COALESCE(a.iata, a.icao, CAST(a.id AS STRING)) AS code,
    a.name,
    a.city,
    a.country,
    a.latitude,
    a.longitude

FROM
    airports AS a

WHERE 1=1
    AND a.id IN (SELECT src FROM edges UNION SELECT dst FROM edges)
""").cache()
vertices.createOrReplaceTempView("vertices")

g = GraphFrame(vertices, edges)

```

	count
vertices	3329
edges	37269

Рисунок 2.13 – Розмір графа авіасполучень

2.5.1 Пошук мотивів

Маршрути з 1, 2 і 3 сегментами знаходить функція `simple_paths` (лістинг 2.7). Для трьох сегментів вона формує мотив $(a) - [e1] \rightarrow (b) ; (b) - [e2] \rightarrow (c) ; (c) - [e3] \rightarrow (d)$ і умову `a.country = 'Germany' AND d.country = 'United Kingdom'` разом із шістьма нерівностями `a.id != b.id, a.id != c.id, ...` для всіх пар вершин. Кількість знайдених маршрутів наведено на рис. 2.14, найкоротші маршрути кожної довжини – на рис. 2.15.

Лістинг 2.7 – Пошук маршрутів мотивами

```
FROM_DE = "{v}.country = 'Germany'"
TO_UK = "{v}.country = 'United Kingdom'"

de_uk = {segments: simple_paths(g, segments, FROM_DE, TO_UK).cache() for
        segments in (1, 2, 3)}
```

routes	
segments	
1	73
2	9178
3	506498

Рисунок 2.14 – Кількість маршрутів з Німеччини до Великої Британії за кількістю сегментів

	segments	route	distance_km
0	1	NRN → STN	407.805873
1	1	DUS → STN	455.938911
2	1	DUS → LCY	466.072330
3	1	DUS → LGW	484.609763
4	1	CGN → STN	492.464215
5	1	DUS → LHR	501.891043
6	1	DTM → STN	509.805233
7	1	CGN → LGW	513.811456
8	1	CGN → LHR	534.220806
9	1	HHN → STN	537.439425

	segments	route	distance_km
0	2	DUS → AMS → NWI	417.511556
1	2	CGN → AMS → NWI	468.809446
2	2	DUS → AMS → SEN	468.980381
3	2	DUS → AMS → STN	491.224808
4	2	DUS → AMS → LCY	513.410833
5	2	CGN → AMS → SEN	520.278271
6	2	BRE → AMS → NWI	522.498312
7	2	DUS → AMS → LTN	532.194424
8	2	CGN → AMS → STN	542.522698
9	2	DUS → AMS → LGW	543.045250

	segments	route	distance_km
0	3	DUS → AMS → NWI → HUY	565.374510
1	3	FRA → DUS → AMS → NWI	606.379648
2	3	CGN → AMS → NWI → HUY	616.672401
3	3	SCN → LUX → AMS → NWI	634.016665
4	3	FMO → BRE → AMS → NWI	648.368344
5	3	FRA → DUS → AMS → SEN	657.848473
6	3	SCN → LUX → CDG → LGW	660.652984
7	3	DUS → AMS → BRU → LGW	664.044294
8	3	DUS → AMS → NWI → MAN	667.129396
9	3	BRE → AMS → NWI → HUY	670.361266

Рисунок 2.15 – Найкоротші маршрути з 1, 2 і 3 сегментами

Як Spark виконує мотив, видно з плану запиту у вебінтерфейсі драйвера (рис. 2.16, мотив з одного сегмента). Таблиця ребер (37 269 рядків) з'єднується з вершинами двічі: для аеропорту вильоту та прильоту. Оптимізатор Catalyst переніс умови на країну до з'єднань: з 3 329 вершин фільтр залишає 32 аеропорти Німеччини та 52 Великої Британії. Ці невеликі таблиці розсилаються на всі executors (BroadcastExchange), тож з'єднання відбу-

вається без перерозподілу (shuffle) таблиці ребер: перше з'єднання залишає 1 409 ребер з Німеччини, друге – 73 прямі маршрути.

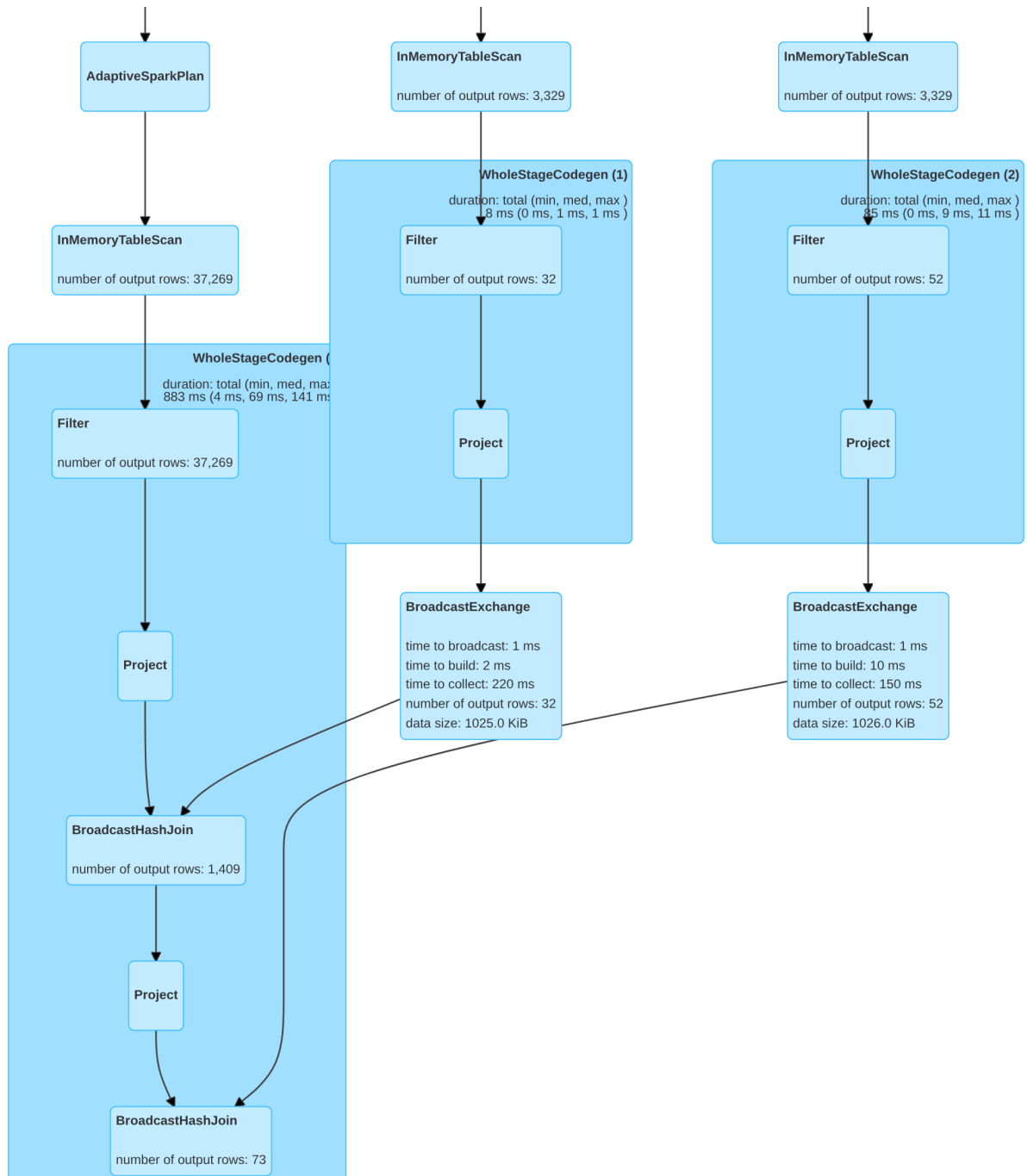


Рисунок 2.16 – Фрагмент плану виконання мотиву з одного сегмента у вебінтерфейсі Spark

Аеропорти пересадки на маршрутах з однією пересадкою наведено на рис. 2.17.

	hub	count
0	PMI	648
1	AGP	391
2	TFS	340
3	AMS	288
4	FAO	286
5	ALC	260
6	DUB	242
7	ACE	240
8	IBZ	240
9	LPA	238

Рисунок 2.17 – Найчастіші аеропорти пересадки (2 сегменти)

2.5.2 Пошук у ширину (BFS)

Той самий пошук методом BFS виконує один виклик (лістинг 2.8). BFS повертає всі шляхи мінімальної довжини (за кількістю сегментів), але мінімум визначається один для всієї множини початкових вершин: пошук іде одночасно від усіх аеропортів Німеччини і зупиняється на першій глибині, на якій досягнуто будь-якого аеропорту Великої Британії. З частини німецьких аеропортів є прямі рейси, тому ця глибина дорівнює 1, і результат – 73 прямі маршрути (рис. 2.18), ті самі, що й у мотиву з одного сегмента.

Лістинг 2.8 – Пошук маршрутів методом BFS

```
bfs = g.bfs(fromExpr="country = 'Germany'", toExpr="country = 'United Kingdom'
            ", maxPathLength=3)
bfs.count(), bfs.columns
```

```
(73, ['from', 'e0', 'to'])
```

Рисунок 2.18 – Результат BFS для Німеччина → Велика Британія

Отже, пошук від усієї Німеччини не показує, як BFS іде глибше, і не відрізняється від мотиву з одного сегмента. Тому BFS запущено ще й від одного аеропорту, з якого немає прямих рейсів до Великої Британії. Запит

обирає такий аеропорт з найбільшою кількістю напрямків, щоб маршрутів з пересадкою було якомога більше: це Фрідріхсгафен (FDH). Для нього мінімальна довжина – 2 сегменти, і BFS повертає 90 маршрутів з однією пересадкою (рис. 2.19). Цих маршрутів немає в результаті пошуку від усієї Німеччини, хоча FDH – німецький аеропорт. Порівняння кількості шляхів, знайдених обома методами, наведено на рис. 2.20; аналіз – у підрозділі 3.2.

	route	distance_km
0	FDH → CGN → STN	887.028479
1	FDH → FRA → LCY	891.905035
2	FDH → DUS → STN	904.395832
3	FDH → CGN → LGW	908.375720
4	FDH → DUS → LCY	914.529252
5	FDH → FRA → LHR	926.272522
6	FDH → CGN → LHR	928.785070
7	FDH → DUS → LGW	933.066684
8	FDH → DUS → LHR	950.347964
9	FDH → CGN → BHX	1032.267664

Рисунок 2.19 – Найкоротші за відстанню з маршрутів BFS для FDH

	BFS	motifs (1-3 segments)
Germany → United Kingdom	73	515749
FDH → United Kingdom	90	6028

Рисунок 2.20 – Кількість шляхів: BFS і мотиви з 1–3 сегментів

2.6 Завдання 3. Найкоротший та найдовший маршрути TXL – EDI

Між Берлін-Тегель (TXL) і Единбургом (EDI) шукаються прості шляхи з 1–4 сегментів (до 3 пересадок) тією ж функцією `simple_paths` (лістинг 2.9); довжина маршруту – сума відстаней сегментів у кілометрах. Прямого рейсу немає; маршрутів з 2, 3 і 4 сегментів знайдено 40, 2 745 і 180 069 (рис. 2.21). Найкоротші та найдовші маршрути наведено на рис. 2.22.

Лістинг 2.9 – Маршрути TXL → EDI з 1–4 сегментів

```

SRC, DST = "TXL", "EDI"

paths = None
for segments in (1, 2, 3, 4):
    found = simple_paths(g, segments, f"{{v}}.code = '{SRC}'", f"{{v}}.code = '{DST}'")
    paths = found if paths is None else paths.unionByName(found)
paths = paths.cache()

show(paths.groupBy("segments").count().orderBy("segments"))

```

	segments	count
0	2	40
1	3	2745
2	4	180069

Рисунок 2.21 – Кількість маршрутів TXL → EDI за кількістю сегментів

	segments	route	distance_km
0	2	TXL → AMS → EDI	1244.550407
1	3	TXL → AMS → NWI → EDI	1290.150128
2	3	TXL → DUS → AMS → EDI	1313.451568
3	2	TXL → CPH → EDI	1343.178733
4	2	TXL → CGN → EDI	1358.983140

	segments	route	distance_km
0	4	TXL → ORD → HKG → EWR → EDI	37808.463810
1	4	TXL → JFK → HKG → EWR → EDI	37542.824146
2	4	TXL → ORD → DEL → EWR → EDI	36111.027015
3	4	TXL → ORD → PVG → EWR → EDI	35519.979974
4	4	TXL → JFK → PVG → EWR → EDI	35346.186868

Рисунок 2.22 – Найкоротші та найдовші маршрути TXL → EDI

Найкоротший маршрут – TXL → AMS → EDI (1 245 км) з пересадкою в Амстердамі. Найдовший – TXL → ORD → HKG → EWR → EDI (37 808 км, 94 % довжини екватора): формально він задовольняє умови (не більше трьох пересадок, жоден аеропорт не повторюється), але практичного сенсу не має.

Найдовші маршрути складаються з трансатлантичних і транстихоокеанських сегментів між найбільшими хабами, бо саме з них є найдовші безпосадкові рейси.

2.7 Завдання 4. PageRank

PageRank запущено з `resetProbability = 0.15`, що відповідає коефіцієнту демпфування 0,85, і 20 ітераціями (лістинг 2.10). Для порівняння до результату додано вхідний і вихідний ступені вершин (рис. 2.23); аеропорти з найбільшою кількістю напрямків показано на рис. 2.24.

Лістинг 2.10 – PageRank і ступенева центральність

```

ranks = g.pageRank(resetProbability=0.15, maxIter=20).vertices
degrees = g.inDegrees.join(g.outDegrees, "id")

show(
  ranks.join(degrees, "id")
    .select("code", "name", "city", "country", F.round("pagerank", 2).alias("
pagerank"), "inDegree", "outDegree")
    .orderBy(F.desc("pagerank")),
  10,
)

```

	code	name	city	country	pagerank	inDegree	outDegree
0	ATL	Hartsfield Jackson Atlanta International Airport	Atlanta	United States	15.95	216	217
1	ORD	Chicago O'Hare International Airport	Chicago	United States	14.63	203	206
2	ISL	Atatürk International Airport	Istanbul	Turkey	14.62	228	225
3	DEN	Denver International Airport	Denver	United States	14.54	168	169
4	DFW	Dallas Fort Worth International Airport	Dallas-Fort Worth	United States	14.29	185	187
5	DME	Domodedovo International Airport	Moscow	Russia	13.72	188	187
6	CDG	Charles de Gaulle International Airport	Paris	France	13.44	233	237
7	FRA	Frankfurt am Main Airport	Frankfurt	Germany	13.08	238	239
8	PEK	Beijing Capital International Airport	Beijing	China	12.97	206	206
9	AMS	Amsterdam Airport Schiphol	Amsterdam	Netherlands	12.40	231	232

Рисунок 2.23 – 10 аеропортів з найбільшим PageRank

	code	name	country	inDegree	outDegree
0	FRA	Frankfurt am Main Airport	Germany	238	239
1	CDG	Charles de Gaulle International Airport	France	233	237
2	AMS	Amsterdam Airport Schiphol	Netherlands	231	232
3	ISL	Atatürk International Airport	Turkey	228	225
4	ATL	Hartsfield Jackson Atlanta International Airport	United States	216	217
5	PEK	Beijing Capital International Airport	China	206	206
6	ORD	Chicago O'Hare International Airport	United States	203	206
7	MUC	Munich Airport	Germany	189	191
8	DFW	Dallas Fort Worth International Airport	United States	185	187
9	DME	Domodedovo International Airport	Russia	188	187

Рисунок 2.24 – 10 аеропортів з найбільшою кількістю напрямків вильоту

Код ISL належить стамбульському аеропорту Ататюрк: у 2019 році код IST перейшов до нового аеропорту Стамбула, і довідник аеропортів OpenFlights це відображає, тоді як маршрути зібрано раніше (2014 рік).

Вплив параметра `resetProbability` досліджено для значень 0,05–0,85 (лістинг 2.11, рис. 2.25).

Лістинг 2.11 – PageRank для різних значень `resetProbability`

```
top_by_reset = {}
for reset in (0.05, 0.15, 0.3, 0.5, 0.85):
    top = g.pageRank(resetProbability=reset, maxIter=20).vertices.orderBy(F.
desc("pagerank")).limit(5)
    top_by_reset[reset] = [f"{row.code} ({row.pagerank:.1f})" for row in top.
collect()]
```

	1	2	3	4	5
resetProbability					
0.05	ATL (17.7)	ISL (16.7)	CDG (16.6)	FRA (16.6)	ORD (16.5)
0.15	ATL (16.0)	ORD (14.6)	ISL (14.6)	DEN (14.5)	DFW (14.3)
0.30	ATL (13.8)	DEN (13.7)	DME (13.1)	DFW (12.8)	ORD (12.5)
0.50	DEN (11.6)	DME (11.5)	ATL (11.0)	DFW (10.5)	ORD (9.9)
0.85	DME (5.2)	DEN (5.1)	ATL (4.6)	DFW (4.6)	ORD (4.2)

Рисунок 2.25 – 5 найвпливовіших аеропортів для різних значень `resetProbability`

У вебінтерфейсі драйвера видно, як GraphFrames виконує алгоритми (рис. 2.26). Для PageRank граф перетворюється на RDD (`rdd at GraphFrame.scala`), ітерації виконує GraphX, а результат повертається у DataFrame (`count at PageRank.scala`); кожен з п'яти запусків з лістингу 2.11 дає дві такі операції. Компоненти зв'язності (`TwoPhase.scala`, наступний підрозділ) реалізовано серією запитів DataFrame із записом проміжних результатів у checkpoint-директорію (`parquet at TwoPhase.scala`).

ID ▼	Description	Submitted	Duration
72	collect at TwoPhase.scala:174	2026/10/08 16:34:34	0.1 s
71	first at TwoPhase.scala:123	2026/10/08 16:34:32	2 s
70	count at TwoPhase.scala:299	2026/10/08 16:34:32	82 ms
69	collect at /tmp/ipykernel_1526770/1188430003.py:4	2026/10/08 16:34:32	65 ms
68	count at PageRank.scala:141	2026/10/08 16:34:32	0.4 s
67	count at PageRank.scala:140	2026/10/08 16:34:31	0.2 s
66	collect at /tmp/ipykernel_1526770/1188430003.py:4	2026/10/08 16:34:26	84 ms
65	count at PageRank.scala:141	2026/10/08 16:34:26	0.5 s
64	count at PageRank.scala:140	2026/10/08 16:34:26	0.2 s
63	collect at /tmp/ipykernel_1526770/1188430003.py:4	2026/10/08 16:34:20	0.1 s
62	count at PageRank.scala:141	2026/10/08 16:34:20	0.5 s
61	count at PageRank.scala:140	2026/10/08 16:34:19	0.5 s
60	collect at /tmp/ipykernel_1526770/1188430003.py:4	2026/10/08 16:34:15	79 ms
59	count at PageRank.scala:141	2026/10/08 16:34:14	0.5 s
58	count at PageRank.scala:140	2026/10/08 16:34:14	0.5 s
57	collect at /tmp/ipykernel_1526770/1188430003.py:4	2026/10/08 16:34:08	0.3 s
56	count at PageRank.scala:141	2026/10/08 16:34:07	0.5 s
55	count at PageRank.scala:140	2026/10/08 16:34:07	0.4 s
54	toPandas at /tmp/ipykernel_1526770/403559947.py:26	2026/10/08 16:33:56	0.4 s
53	toPandas at /tmp/ipykernel_1526770/403559947.py:26	2026/10/08 16:29:38	0.9 s
52	count at PageRank.scala:141	2026/10/08 16:29:37	1 s
51	count at PageRank.scala:140	2026/10/08 16:29:35	1 s
50	rdd at GraphFrame.scala:348	2026/10/08 16:29:23	47 ms
49	rdd at GraphFrame.scala:340	2026/10/08 16:29:23	97 ms

Рисунок 2.26 – Запити PageRank і Connected Components у вкладці SQL / DataFrame вебінтерфейсу Spark

2.8 Завдання 5. Кластеризація та аналіз зв'язності

Слабко зв'язні компоненти обчислює метод `connectedComponents` (лістинг 2.12). Алгоритм потребує `checkpoint`-директорії, спільної для драйвера та всіх `executors`; її розміщено в змонтованій директорії `data/`. Розміри всіх компонент наведено на рис. 2.27, кластери з не менше ніж 5 аеропортів – на рис. 2.28.

Лістинг 2.12 – Компоненти зв'язності

```
components = g.connectedComponents().cache()
component_sizes = components.groupBy("component").count().orderBy(F.desc("count"))
show(component_sizes)

show(component_sizes.filter("count >= 5"))
```

	component	count
0	1	3303
1	1998	10
2	3726	4
3	5642	4
4	3860	4
5	7309	2
6	6448	2

Рисунок 2.27 – Розміри компонент зв'язності

	component	count
0	1	3303
1	1998	10

Рисунок 2.28 – Компоненти з не менше ніж 5 аеропортів

Ізольовані підграфи – компоненти з 2 і більше аеропортів, крім найбільшої (основної мережі) – знаходить запит з лістингу 2.13 (рис. 2.29).

Лістинг 2.13 – Ізольовані підграфи

```
main_component = component_sizes.first().component
isolated = (
    components.join(component_sizes, "component")
    .filter((F.col("component") != main_component) & (F.col("count") >= 2))
    .groupBy("component", "count")
    .agg(
        F.concat_ws(" ", F.sort_array(F.collect_list(F.concat_ws(" ", "code",
        F.concat(F.lit("("), "city", F.lit(")"))))))).alias("airports"),
        F.concat_ws(" ", F.array_distinct(F.collect_list("country"))).alias("
countries"),
    )
    .orderBy(F.desc("count"))
)
```

	component	count	airports	countries
0	1998	10	BMY (Waala), GEA (Noumea), ILP (Île des Pins), KNQ (Kone), KOC (Koumac), LIF...	New Caledonia
1	5642	4	ERS (Windhoek), MPA (Mpacha), NDU (Rundu), OND (Ondangwa)	Namibia
2	3860	4	AKB (Atka), DUT (Unalaska), IKO (Nikolski), KQA (Akutan)	United States
3	3726	4	BFI (Seattle), CLM (Port Angeles), ESD (Eastsound), FRD (Friday Harbor)	United States
4	7309	2	SPB (Charlotte Amalie), SSB (Christiansted)	Virgin Islands
5	6448	2	BLD (Boulder City), GCW (Grand Canyon West)	United States

Рисунок 2.29 – Ізольовані підграфи

Для виділення кластерів усередині основної мережі запущено Label Propagation з 5 ітераціями (лістинг 2.14); він знайшов 71 спільноту з не менше ніж 5 аеропортів (рис. 2.30). Для кожної спільноти обчислено розмір, кількість країн, головну країну з її часткою та п'ять аеропортів з найбільшою кількістю напрямків (рис. 2.31).

Лістинг 2.14 – Label Propagation

```
communities = g.labelPropagation(maxIter=5).cache()
community_sizes = communities.groupBy("label").count()
community_sizes.filter("count >= 5").count()
```

Рисунок 2.30 – Кількість спільнот з не менше ніж 5 аеропортів

	label	airports	countries	main_country	main_airports	main_country_share
0	193	796	85	United States	ISL, ATL, DXB, JFK, EWR	0.43
1	16	351	50	Norway	FRA, CDG, AMS, MUC, LHR	0.12
2	302	336	64	France	BCN, MAD, PMI, AGP, MXP	0.10
3	340	216	33	India	DME, BKK, VKO, SVX, TAS	0.25
4	156	159	20	Japan	PEK, PVG, CAN, HKG, ICN	0.35
5	5	157	17	Australia	SIN, KUL, SYD, MNL, CGK	0.46
6	49	102	3	United States	ORD, DFW, DEN, IAH, LAX	0.49
7	3364	85	3	China	WUX, TXN, CZX, XUZ, MIG	0.98
8	2531	73	1	Brazil	BSB, CNF, SSA, CGH, CWB	1.00
9	2276	70	9	China	URC, CGO, KWE, FOC, NNG	0.63
10	813	49	13	Madagascar	DAR, TNR, MPM, RUN, MBA	0.27
11	3320	40	3	Australia	NOU, ISA, KTA, LST, MKY	0.95
12	3599	38	1	United States	BET, KSM, RSH, KWK, EMK	1.00
13	248	33	10	Congo (Kinshasa)	JNB, NBO, CPT, LUN, DUR	0.24
14	2397	31	2	Philippines	MUR, RPML, PPS, BCD, ZAM	0.97

Рисунок 2.31 – 15 найбільших спільнот Label Propagation

3 АНАЛІЗ РЕЗУЛЬТАТІВ

3.1 Авіакомпанія з найбільшим кілометражем

П'ять авіакомпаній з найбільшою сумарною довжиною власних маршрутів наведено в табл. 3.1; середню довжину маршруту обчислено як відношення сумарної довжини до кількості маршрутів.

Таблиця 3.1 – Авіакомпанії з найбільшою сумарною довжиною маршрутів

Авіакомпанія	Країна	Маршрутів	Сумарно, км	Середня, км
Ryanair	Ірландія	2 484	3 700 425	1 490
US Airways	США	1 446	3 477 460	2 405
American Airlines	США	1 265	3 342 406	2 642
United Airlines	США	905	3 046 248	3 366
Delta Air Lines	США	1 146	3 013 769	2 630

Найбільший кілометраж має Ryanair: 3,70 млн км на 2 484 маршрутах. Лідерство забезпечує не довжина рейсів (у середньому 1 490 км – найменше серед п'ятірки), а кількість маршрутів: лоукостер будує мережу «з точки в точку» між сотнями європейських аеропортів, і майже кожна пара його баз має окремий маршрут. Американські мережеві перевізники мають у 1,7–2,7 рази менше маршрутів, але вони довші (2 405–3 366 км) завдяки трансконтинентальним і міжнародним рейсам.

Найменший кілометраж має TransHolding System (Бразилія): 36 км на 2 маршрутах. Усі 10 авіакомпаній з найменшим кілометражем мають рівно 2 власні маршрути загальною довжиною 36–237 км; серед них вертолітні (Papillon Grand Canyon Helicopters, Helijet) та місцеві перевізники.

Метрика рахує кожен маршрут один раз, тому показує довжину мережі маршрутів, а не реальний наліт: частоти рейсів у OpenFlights відсутні. Codeshare-маршрути (14 597) не враховано, інакше авіакомпанії отримували б кілометри рейсів партнерів.

3.2 Порівняння пошуку мотивів і BFS

Результати обох методів наведено в табл. 3.2 (для FDH у ноутбучі обчислено лише загальну кількість маршрутів мотивами), порівняння реалізації – у табл. 3.3.

Таблиця 3.2 – Кількість знайдених маршрутів

Пошук	Мотиви				BFS
	1 сегм.	2 сегм.	3 сегм.	Разом	
Німеччина → Велика Британія	73	9 178	506 498	515 749	73
FDH → Велика Британія	0	–		6 028	90

Результати. Мотиви знаходять 73 прямі маршрути, 9 178 з однією пересадкою і 506 498 з двома. Кожен додатковий сегмент збільшує кількість маршрутів у 55–126 разів: у графі в середньому 11,2 напрямку з аеропорту, а через хаби з сотнями напрямків число продовжень зростає ще швидше. BFS повертає всі маршрути з мінімальною кількістю сегментів, причому мінімум один для всієї множини початкових аеропортів. Від усієї Німеччини це 1 сегмент, тому BFS дає ті самі 73 прямі маршрути, що й мотив з одного сегмента; маршрути з пересадкою до результату не потрапляють навіть для аеропортів без прямих рейсів, таких як FDH. Щоб порівняти методи на однаковій множині шляхів, обидва запущено від одного аеропорту FDH: його мінімум – 2 сегменти, і BFS знаходить 90 маршрутів з однією пересадкою, тоді як мотиви з 1–3 сегментів – 6 028 маршрутів з однією або двома пересадками (прямих рейсів з FDH немає). Отже, BFS відповідає на питання «як дістатися з мінімальною кількістю пересадок», а всі маршрути до двох пересадок, яких вимагає завдання, дають лише мотиви.

Складність реалізації. BFS – один виклик бібліотеки з умовами на початкові й кінцеві вершини. Мотиви потребують окремого шаблону для

Таблиця 3.3 – Порівняння реалізації пошуку мотивів і BFS

Критерій	Мотиви	BFS
Що знаходить	усі шляхи заданої довжини	усі шляхи мінімальної кількості сегментів
Виклики	окремий мотив для кожної довжини (1, 2, 3)	один виклик з <code>maxPathLength=3</code>
Прості шляхи	явні умови $u \neq v$ для кожної пари вершин: 1, 3, 6 умов	гарантовано: найкоротший шлях не повторює вершин
Кінцеві вершини	умови на першу та останню вершину	<code>fromExpr, toExpr</code>
Вартість	$2k$ з'єднань для k сегментів; результат зростає комбінаторно	з'єднання по одному рівню, зупинка на першій глибині з результатом
Відстань маршруту	сума атрибутів ребер $e_1..e_k$	можлива, але лише для найкоротших шляхів

кожної довжини шляху і явних фільтрів: `GraphFrames` не забороняє повторювати вершину в мотиві, тому без умов $a \neq c$ та інших знайшлися б маршрути на кшталт `DUS → AMS → DUS → LHR`. Функція `simple_paths` (додаток Б) генерує ці шаблони та умови, але логіка «простого шляху» – відповідальність користувача, а не бібліотеки. Обчислювальна вартість також різна: мотив з k сегментів – це $2k$ з'єднань (рис. 2.16), і результат для трьох сегментів (506 498 рядків) необхідно повністю матеріалізувати, тоді як BFS зупиняється, щойно знайдено ціль.

Пересадки. Найчастіші аеропорти пересадки – курортні аеропорти Іспанії та Португалії: Пальма-де-Мальорка (648 маршрутів), Малага (391),

Тенерифе-Південь (340), а також Амстердам (288). Лоукостери літають на курорти з багатьох аеропортів обох країн, тому граф містить багато пар «Німеччина → курорт → Велика Британія». Пошук у графі не враховує розкладу та авіакомпаній, тому такі маршрути – можливі комбінації рейсів, а не квитки з пересадкою, які продають перевізники.

3.3 Найвпливовіші аеропорти за PageRank

П'ять найвпливовіших аеропортів за PageRank (`resetProbability = 0.15`): Атланта (ATL, 15,95), Чикаго О'Хара (ORD, 14,63), Стамбул Ататюрк (ISL, 14,62), Денвер (DEN, 14,54) та Даллас/Форт-Ворт (DFW, 14,29). Значення PageRank у GraphFrames нормовано так, що їх сума дорівнює кількості вершин (середнє – 1), тобто ранг Атланти приблизно в 16 разів більший за середній.

Чому саме вони. PageRank аеропорту – сума часток рангу аеропортів, з яких до нього є рейси, причому кожен аеропорт ділить свій ранг порівну між усіма своїми напрямками. Американські хаби (ATL, ORD, DEN, DFW) отримують ранг від сотень регіональних аеропортів США, для яких хаб – один з небагатьох напрямків, тож вони віддають йому значну частку рангу. Крім того, хаби США густо пов'язані між собою і передають ранг один одному. Стамбул поєднує велику кількість напрямків (228 вхідних) з роллю транзитного вузла між Європою, Азією та Африкою.

Порівняння зі ступеневою центральністю. За кількістю напрямків лідирують FRA (239), CDG (237) і AMS (232), а за PageRank вони лише на 7–10 місцях (рис. 2.23, 2.24). Європейські хаби мають багато напрямків, але значна частина їхніх сусідів – інші великі хаби, які ділять свій ранг між сотнями напрямків, тож кожне ребро приносить мало рангу. Різниця показує, що PageRank оцінює не кількість, а «якість» зв'язків.

Порівняння з найбільшими хабами світу. Маршрути OpenFlights відповідають 2014 року, тому для порівняння взято рейтинг Airports Council International (ACI) за пасажиропотоком 2014 року [6] (табл. 3.4).

Таблиця 3.4 – Найбільші аеропорти світу за пасажиропотоком 2014 року (ACI) та їх місце за PageRank

Місце ACI	Код	Аеропорт	Пасажирів, млн	Місце за PageRank
1	ATL	Атланта	96,2	1
2	PEK	Пекін	86,1	9
		Столичний		
3	LHR	Лондон Хітроу	73,4	поза 10
4	HND	Токіо Ханеда	72,8	поза 10
5	LAX	Лос-Анджелес	70,7	поза 10
6	DXB	Дубай	70,5	поза 10
7	ORD	Чикаго О'Хара	70,0	2
8	CDG	Париж Шарль-де-Голль	63,8	7
9	DFW	Даллас/Форт-Ворт	63,6	5
10	HKG	Гонконг	63,1	поза 10

Збіг частковий. З п'ятірки PageRank до десятки ACI входять ATL (1), ORD (7) і DFW (9). Стамбул Ататюрк у 2014 році був 13-м за пасажиропотоком [7], Денвер до першої десятки не входив. Водночас Хітроу, Ханеда, Лос-Анджелес, Дубай і Гонконг – 3–6 і 10 місця ACI – не входять навіть до першої десятки за PageRank, а Хітроу – і до десятки за кількістю напрямків (у нього менше ніж 187).

Розбіжність зумовлена тим, що саме вимірює PageRank на цьому графі. Граф містить одне ребро на пару аеропортів, без частоти рейсів, місткості

літаків і кількості пасажирів, тому PageRank оцінює мережу напрямків, а не пасажиропотік. Щоденні рейси A380 між Лондоном і Дубаєм і рейс невеликого літака раз на тиждень мають однакову вагу. Тому вгору піднімаються аеропорти з густою внутрішньою мережею США, де хаб сполучений з сотнями регіональних аеропортів. Аеропорти з обмеженою кількістю слотів (LHR, HND) обслуговують величезну кількість пасажирів на меншій кількості напрямків – частими рейсами великих літаків, а Ханеда у 2014 році обслуговувала переважно внутрішні рейси Японії. PageRank таких аеропортів нижчий, хоча за пасажиропотоком вони серед найбільших. Отже, PageRank на графі OpenFlights виявляє вузли, найважливіші для зв'язності мережі напрямків, і для оцінки хабів за пасажиропотоком потрібен граф з вагами ребер (частота рейсів або кількість місць).

Вплив resetProbability. Зі зростанням імовірності скидання значення PageRank вирівнюються: ранг лідера зменшується з 17,7 при 0,05 до 5,2 при 0,85, бо більша частина рангу розподіляється рівномірно між усіма вершинами. Склад лідерів також змінюється (рис. 2.25). При малих значеннях (0,05) ранг поширюється по довгих ланцюжках, і до п'ятірки входять великі міжнародні хаби CDG та FRA – оцінка глобальна. При великих значеннях (0,5–0,85) ранг вершини визначає переважно її найближче оточення, і лідерами стають DME, DEN, ATL – аеропорти з великою кількістю регіональних напрямків, для яких вони є одним із небагатьох сполучень (Домоєдово для аеропортів Росії та СНД, Денвер для регіону Скелястих гір). Значення 0,15 – компроміс, за якого враховується і транзитивний вплив, і локальна структура; при цьому ATL лідирує для значень 0,05–0,3.

3.4 Кластери та ізольовані підграфи

Компоненти зв'язності. Мережа майже повністю зв'язна: 3 303 з 3 329 аеропортів (99,2 %) утворюють одну слабо зв'язну компоненту. Решта 26 аеро-

портів розподілені на 6 ізольованих груп (табл. 3.5); одиночних ізольованих аеропортів немає, бо вершинами графа є лише аеропорти з маршрутами. Умову «не менше 5 аеропортів» задовольняють лише дві компоненти: основна мережа та група Нової Каледонії (10 аеропортів).

Таблиця 3.5 – Ізольовані підграфи

Регіон	Аеропортів	Аеропорти
Нова Каледонія	10	GEA (Нумеа), острови: BMY, ILP, KNQ, KOC, LIF та ін.
Намібія	4	ERS (Віндгук), MPA, NDU, OND
Алеутські острови (Аляска, США)	4	AKB, DUT, IKO, KQA
Сіетл та острови Сан-Хуан (США)	4	BFI, CLM, ESD, FRD
Американські Віргінські острови	2	SPB, SSB
Невада – Гранд-Каньйон (США)	2	BLD, GCW

Ізольовані групи – місцеві мережі, що мають лише внутрішні рейси: острівні сполучення (Нова Каледонія, Алеутські та Віргінські острови), регіональні рейси Намібії, туристичні рейси до Гранд-Каньйону. У Нумеа та Віндгуку внутрішні рейси виконуються з другого, «внутрішнього» аеропорту міста (GEA, ERS), а міжнародні – з іншого, тому в графі ці мережі відірвані від основної, хоча пасажир може пересісти, переїхавши між аеропортами: міжнародний аеропорт Нумеа NOU належить основній компоненті (див. спільноту з NOU на рис. 2.31). Частина ізольованості може бути й наслідком

неповноти бази OpenFlights: граф містить лише маршрути, які є в ній на 2014 рік.

Спільноти Label Propagation. Оскільки основна компонента охоплює 99% аеропортів, кластери всередині неї виділено алгоритмом Label Propagation: 71 спільнота з не менше ніж 5 аеропортів. Найбільші з них (табл. 3.6) відповідають регіональним мережам: міжнародним хабам разом з частиною мережі США (796 аеропортів), Північній і Центральній Європі (351), Південній Європі з курортами Середземномор'я (336), Росії, СНД і Південній Азії (216), Східній Азії (159), Південно-Східній Азії та Океанії (157), внутрішній мережі США (102), регіональним аеропортам Китаю (85) та внутрішній мережі Бразилії (73).

Таблиця 3.6 – Найбільші спільноти Label Propagation

Аеропортів	Країн	Головна країна (частка)	Головні аеропорти
796	85	США (0,43)	ISL, ATL, DXB, JFK, EWR
351	50	Норвегія (0,12)	FRA, CDG, AMS, MUC, LHR
336	64	Франція (0,10)	BCN, MAD, PMI, AGP, MXP
216	33	Індія (0,25)	DME, BKK, VKO, SVX, TAS
159	20	Японія (0,35)	PEK, PVG, CAN, HKG, ICN
157	17	Австралія (0,46)	SIN, KUL, SYD, MNL, CGK
102	3	США (0,49)	ORD, DFW, DEN, IAH, LAX
85	3	Китай (0,98)	WUX, TXN, CZX, XUZ, MIG
73	1	Бразилія (1,00)	BSB, CNF, SSA, CGH, CWB

Спільноти мають чітку географічну основу: менші з них майже повністю належать одній країні (Китай 98 %, Бразилія 100 %, Аляска – спільнота з ВЕТ, 100 % США), а великі об'єднують регіони навколо спільних хабів. Розбиття не збігається з межами країн: аеропорти США потрапили до трьох спільнот (глобальної з ATL і JFK, внутрішньої з ORD і DFW, аляскинської), а Європа поділена на «північну» мережу мережевих перевізників (FRA, AMS, LHR) і «південну» курортну мережу лоукостерів (BCN, PMI, AGP). Label Propagation виконано з фіксованою кількістю ітерацій (тут 5), без гарантії збі-

жності, а нічий між мітками розв’язуються довільно, тому межі між великими спільнотами наближені; стабільним є саме поділ на регіональні мережі.

ВИСНОВКИ

У роботі розгорнуто кластер Apache Spark 4.0.4 (master і два робочі вузли) у Docker і з бібліотекою GraphFrames проаналізовано мережу авіасполучень OpenFlights: 67 663 маршрути, 12 668 аеропортів і 6 162 авіакомпанії.

Підготовка даних з’єднанням маршрутів з аеропортами дала координати для 67 235 маршрутів. Перевірка найдовших відстаней виявила пошкоджений запис аеропорту 5613 (координати Каліфорнії для аеропорту в Замбії), маршрути через який виключено. Найбільшу сумарну довжину власних маршрутів має Ryanair (3,70 млн км на 2 484 маршрутах) – за рахунок кількості маршрутів, а не їх довжини; найменшу – TransHolding System (36 км на 2 маршрутах).

Граф з 3 329 аеропортів і 37 269 напрямків дозволив знайти маршрути з Німеччини до Великої Британії: мотиви знаходять 73 прямі маршрути, 9 178 з однією пересадкою і 506 498 з двома, тоді як BFS – лише 73 найкоротші. BFS простіший у використанні (один виклик), але відповідає лише на питання про мінімальну кількість пересадок; мотиви потребують шаблону для кожної довжини та явних умов простого шляху, і їхній результат зростає комбінаторно. Між TXL та EDI прямого рейсу немає; найкоротший маршрут TXL → AMS → EDI має 1 245 км, найдовший з не більше ніж трьома пересадками – 37 808 км.

Найвпливовіші за PageRank аеропорти – ATL, ORD, ISL, DEN, DFW. Вони лише частково збігаються з найбільшими аеропортами світу за пасажиропотоком 2014 року: граф без ваг оцінює мережу напрямків, тому вгору піднімаються хаби густої внутрішньої мережі США, а аеропорти з обмеженою кількістю слотів і великим пасажиропотоком (LHR, HND) мають нижчий

ранг. Зі зростанням `resetProbability` значення PageRank вирівнюються, а лідерами стають аеропорти з багатьма регіональними напрямками (DME, DEN).

Аналіз зв'язності показав, що 99,2 % аеропортів утворюють одну компоненту; 6 ізольованих груп (Нова Каледонія, Намібія, Алеутські острови та ін.) мають лише місцеві рейси. Label Propagation виділив 71 спільноту з не менше ніж 5 аеропортів, що відповідають регіональним мережам. Вебінтерфейс Spark підтвердив, що GraphFrames виконує графові алгоритми як запити DataFrame: мотив – як ланцюжок broadcast-з'єднань з перенесеними оптимізатором фільтрами, компоненти зв'язності – як ітеративні запити з checkpoint-ами, а PageRank – через GraphX.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- [1] OpenFlights. «Airport, airline and route data», дата зверн. 8 жовт. 2026. url: <https://openflights.org/data.php>.
- [2] J. Patokallio. «OpenFlights: source code and data (commit 7d1a611)», дата зверн. 8 жовт. 2026. url: <https://github.com/jpatokal/openflights/tree/7d1a611e070295dba776d6afb86e57d0d1aa1cef/data>.
- [3] The Apache Software Foundation. «Spark 4.0.4 Documentation: Spark Standalone Mode», дата зверн. 8 жовт. 2026. url: <https://spark.apache.org/docs/4.0.4/spark-standalone.html>.
- [4] GraphFrames contributors. «GraphFrames User Guide», дата зверн. 8 жовт. 2026. url: <https://graphframes.io/04-user-guide/01-creating-graphframes.html>.

- [5] M. Needham and A. E. Hodler, *Graph Algorithms: Practical Examples in Apache Spark and Neo4j*. Sebastopol, CA: O'Reilly Media, 2019, ISBN: 978-1-492-04768-1.
- [6] BusinessTech. «The busiest airports in the world revealed (ACI World Airport Traffic Report 2014)», дата зверн. 8 жовт. 2026. url: <https://businesstech.co.za/news/general/97223/the-busiest-airports-in-the-world-revealed/>.
- [7] Business Travel News. «BTNews, 13 April 2015: ACI World preliminary world airport traffic and rankings for 2014», дата зверн. 8 жовт. 2026. url: <https://www.btnews.co.uk/article/8668>.

ДОДАТОК А. ПРОГРАМНИЙ КОД

Повний програмний код лабораторної роботи розміщено в репозиторії <https://github.com/nuinashco/iasa-bigdata>, директорія `labs/002-spark-graphframes`:

- `draft.ipynb` – ноутбук Jupyter з усіма кроками роботи та результатами виконання;
- `src/lab002/paths.py` – пошук простих шляхів мотивами (додаток Б);
- `src/lab002/spark.py` – налаштування сесії Spark (драйвер у процесі ноутбука, пакет `GraphFrames`, `checkpoint`-директорія);
- `src/lab002/cluster.py` – запуск кластера та виконання команд;
- `scripts/download_data.py` – завантаження даних `OpenFlights` (додаток Б);
- `infra` – конфігурація кластера (`Docker Compose`, образ Spark з Python 3.13).

Ноутбук відтворює роботу повністю: запуск «Run All» розгортає кластер, завантажує дані та виконує всі обчислення.

ДОДАТОК Б. ДОПОМІЖНІ МОДУЛІ

Лістинг Б.1 – Модуль `paths.py`: пошук простих шляхів мотивами

```
from graphframes import GraphFrame
from pyspark.sql import Column, DataFrame
from pyspark.sql import functions as F

# Vertex aliases of the motif, so a path can have at most len(NODE_NAMES) - 1
# = 7 segments.
NODE_NAMES = "abcdefgh"
```

```

def motif(segments: int) -> str:
    """Chain motif with `segments` edges: 2 -> "(a)-[e1]->(b); (b)-[e2]->(c)".
    """
    nodes = NODE_NAMES[: segments + 1]
    return "; ".join(f"({u})-[e{i}]->({v})" for i, (u, v) in enumerate(zip(
nodes, nodes[1:]), start=1))

def simple_paths(g: GraphFrame, segments: int, start: str, end: str) ->
DataFrame:
    """Routes of exactly `segments` flights that visit no airport twice.

    `start` and `end` are SQL conditions on the first and last airport, with
    `{v}` standing for
    the airport's alias, e.g. "{v}.country = 'Germany'". Vertices need `code`
    and edges
    `distance_km`. Returns segments, route ("TXL → AMS → EDI") and total
    distance_km.
    """
    nodes = NODE_NAMES[: segments + 1]
    distinct = [f"{u}.id != {v}.id" for i, u in enumerate(nodes) for v in
nodes[i + 1:]]
    condition = " AND ".join([start.format(v=nodes[0]), end.format(v=nodes
[-1]), *distinct])
    distance: Column = sum((F.col(f"e{i}").distance_km" for i in range(1,
segments + 1)), F.lit(0.0))
    return g.find(motif(segments)).filter(condition).select(
        F.lit(segments).alias("segments"),
        F.concat_ws(" → ", *(F.col(f"{v}.code" for v in nodes)).alias("route
"),
        distance.alias("distance_km"),
    )

```

Лістинг Б.2 – Скрипт download_data.py

```

"""Download the OpenFlights datasets used in lab 2 into data/openflights/.

Files come from the OpenFlights GitHub repository, pinned to one commit so
every run gets the
same data. Does nothing for files that already exist, unless --force is given.

```

Usage:

```
python labs/002-spark-graphframes/scripts/download_data.py [--data-dir DIR] [--force]
```

"""

```
import argparse
```

```
import urllib.request
```

```
from pathlib import Path
```

```
COMMIT = "7d1a611e070295dba776d6afb86e57d0d1aa1cef"
```

```
BASE_URL = f"https://raw.githubusercontent.com/jpatokal/openflights/{COMMIT}/data"
```

```
FILES = ["airports-extended.dat", "airlines.dat", "routes.dat"]
```

```
DEFAULT_DATA_DIR = Path(__file__).resolve().parent.parent / "data"
```

```
def main() -> None:
```

```
    parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
```

```
    parser.add_argument("--data-dir", type=Path, default=DEFAULT_DATA_DIR)
```

```
    parser.add_argument("--force", action="store_true", help="re-download even if files exist")
```

```
    args = parser.parse_args()
```

```
    target = args.data_dir / "openflights"
```

```
    target.mkdir(parents=True, exist_ok=True)
```

```
    for name in FILES:
```

```
        path = target / name
```

```
        if path.exists() and not args.force:
```

```
            print(f"[skip] {path.name} already present")
```

```
            continue
```

```
        # Download under another name so an interrupted run leaves no truncated file that the
```

```
        # exists() check above would then skip.
```

```
        partial = path.with_suffix(".part")
```

```
        urllib.request.urlretrieve(f"{BASE_URL}/{name}", partial)
```

```
        partial.rename(path)
```

```
        lines = sum(1 for _ in path.open(encoding="utf-8"))
```

```
        print(f"[download] {path.name}: {lines:,} lines, {path.stat().st_size  
/ 1e6:.1f} MB")  
  
if __name__ == "__main__":  
    main()
```